Hybrid Image Encryption Using RSA and AES

Smai Abdelmalek Hadiouche Azouaou

Practical Work — Prof. A.K. Oudjida, NHSM, Sidi-Abdellah — May 2026

1. Introduction

This report presents a hybrid image encryption system implemented in C. We build three encryption pipelines, test them on two images a 720×720 colored butterfly and a 254×255 black-and-white butterfly and evaluate their performance and cryptographic strength.

The three tasks are:

1. **Task 1 — Raw RSA:** Direct block by block RSA encryption of pixel data. Each block of $k-1$ bytes is encrypted as $c = m^e \bmod n$.
2. **Task 2 — RSA + OAEP:** OAEP padding is applied before RSA, introducing randomness for semantic security. The usable block size decreases to $k - 2h - 2$ bytes ($h = 32$ for SHA-256).
3. **Task 3 — Hybrid:** A random AES-256 key is encrypted once with RSA+OAEP; the bulk image is encrypted with AES-256-CBC. This is the standard approach for today's encryption.

Since the test images are provided as PNG (compressed), we convert to BMP for pixel-level security analysis computing histograms on compressed PNG bytes would measure properties of the DEFLATE stream, not the actual pixels. Both PNG and BMP are processed through all three tasks for comprehensive results.

2. Implementation

Our system is organized into four modules, each with a clean C header interface:

Bignum library (`bignum.h`) — Arbitrary-precision arithmetic over 32-bit limbs. Provides addition, subtraction, schoolbook multiplication, restoring division, modular exponentiation, Extended Euclidean GCD, modular inverse, Miller-Rabin primality testing (40 rounds), and CSPRNG-backed prime generation via `RAND_bytes` offered by OpenSSL the bignum structure is defined as:

```c
typedef struct {
  uint32_t limbs[BN_MAX_LIMBS];
  int len, sign;
} bignum_t;
```

RSA (`rsa.h`) — Key generation computes $n = pq$, $e = 65537$, $d = e^{-1} \bmod \varphi(n)$, and CRT parameters $d_p, d_q, q^{-1} \bmod p$. Decryption uses the Chinese Remainder Theorem for 4× speedup:

$$m_1 = c^{d_p} \bmod p, \quad m_2 = c^{d_q} \bmod q$$

$$m = m_2 + q \cdot [q^{-1}(m_1 - m_2) \bmod p]$$

```c
typedef struct {
  bignum_t n, e, d, p, q;
  bignum_t dp, dq, qinv;  // CRT params
  int bits;
} rsa_key_t;
```

OAEP (`oaep.h`) — PKCS#1 v2.2 padding using SHA-256 and MGF1.
AES-CBC (`aes_cbc.h`) — OpenSSL wrapper for AES-256-CBC with PKCS#7 padding.

All RSA arithmetic is hand implemented; only AES and SHA-256 (for OAEP/MGF1) use OpenSSL.

2.1. OAEP Padding Scheme

OAEP transforms deterministic RSA into a semantically secure scheme by injecting randomness. Given message M, key size k bytes, and hash length h:

OAEP Encode(M, k):
1. Compute lHash = SHA256()
2. Build DB = lHash ‖ $\underbrace{00\ldots00}_{\text{PS}}$ ‖ 01 ‖ M *($k-h-1$ bytes)*
3. Generate random seed $r \in \{0,1\}^h$
4. maskedDB = DB ⊕ MGF1(r, |DB|)
5. maskedSeed = r ⊕ MGF1(maskedDB, h)
6. Output EM = 00 ‖ maskedSeed ‖ maskedDB

MGF1 stretches a seed into an arbitrary-length mask by hashing seed ‖ counter. The double-masking ensures identical messages produce different ciphertexts each time, since the random seed r changes on every call.

```c
int oaep_encode(const uint8_t *msg, size_t
msg_len, size_t k, uint8_t *em)
{
  size_t max_msg = OAEP_MAX_MSG_LEN(k);
  if (msg_len > max_msg) {
    fprintf(stderr, "oaep_encode: message too long
    (%zu > %zu)\n",
      msg_len, max_msg);
    return -1;
  }

  size_t db_len = k - OAEP_HASH_LEN - 1;
  uint8_t *db = (uint8_t *)calloc(db_len, 1);
  uint8_t seed[OAEP_HASH_LEN];

  /* DB = lHash || PS(zeros) || 0x01 || M */
  lhash(db);
  /* PS is already zeros from calloc */
  size_t ps_len = db_len - OAEP_HASH_LEN - 1 -
  msg_len;
  db[OAEP_HASH_LEN + ps_len] = 0x01;
  memcpy(db + OAEP_HASH_LEN + ps_len + 1, msg,
  msg_len);

  /* Generate random seed */
  RAND_bytes(seed, OAEP_HASH_LEN);

  /* maskedDB = DB ⊕ MGF1(seed) */
  uint8_t *db_mask = (uint8_t *)malloc(db_len);
  mgf1_sha256(seed, OAEP_HASH_LEN, db_mask, db_len);
  for (size_t i = 0; i < db_len; i++)
    db[i] ^= db_mask[i];
```

```
29
30    /* maskedSeed = seed ⊕ MGF1(maskedDB) */
31    uint8_t seed_mask[OAEP_HASH_LEN];
32    mgf1_sha256(db, db_len, seed_mask, OAEP_HASH_LEN);
33    for (size_t i = 0; i < OAEP_HASH_LEN; i++)
34      seed[i] ^= seed_mask[i];
35
36    /* EM = 0x00 || maskedSeed || maskedDB */
37    em[0] = 0x00;
38    memcpy(em + 1, seed, OAEP_HASH_LEN);
39    memcpy(em + 1 + OAEP_HASH_LEN, db, db_len);
40
41    free(db);
42    free(db_mask);
43    return 0;
44  }
```

3. Performance Analysis

3.1. Theoretical Complexity

Task	Encrypt	Decrypt	Blocks
1: Raw RSA	$O(Nk^2 \log e)$	$O(Nk^2 \log d)$	$N = \lceil \frac{n}{k-1} \rceil$
2: RSA+OAEP	$O(N'k^2 \log e)$	$O(N'k^2 \log d)$	$N' = \lceil \frac{n}{k-66} \rceil$
3: Hybrid	$O(n) + O(k^2 \log e)$	$O(n) + O(k^2 \log d)$	1 RSA + AES

Table 1: Complexity (n=data size, k=128 bytes for 1024-bit RSA).

Tasks 1–2 perform N modular exponentiations per image, each costing $O(k^2 \log e)$ with schoolbook multiplication. Task 3 performs exactly **one** RSA operation (on the 32 byte AES key) plus a linear term with hardware acceleration.

3.2. Measured Execution Times

We test on both PNG file bytes and BMP raw pixels.

Task	Enc (ms)	Dec (ms)	Enc (ms)	Dec (ms)
	Colored (720×720)		*B&W (254×255)*	
PNG file bytes				
1: Raw RSA	5 982	162 886	2 981	81 067
2: RSA+OAEP	11 426	339 896	6 053	166 526
3: Hybrid	**6**	**155**	**7**	**153**
BMP raw pixels				
1: Raw RSA	60 824	1 811 840	8 196	221 715
2: RSA+OAEP	129 867	3 734 811	17 059	460 843
3: Hybrid	**7**	**148**	**7**	**154**

Table 2: Performance comparison (1024-bit RSA). Task 3 times are nearly identical across all inputs.

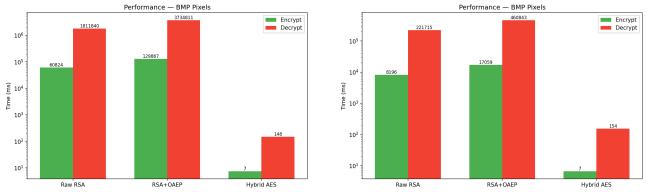

Figure 1: BMP performance (log scale): colored (left) vs B&W (right).

Discussion. Task 3 is 20 000× faster than Task 1 on the colored BMP. For Tasks 1–2, execution time scales linearly with data size: the colored BMP (1.55 MB) takes 10× longer than the B&W BMP (193 KB), which directly matches the 10× difference in pixel count. Task 3 remains constant at 7 ms encrypt / 150 ms decrypt regardless of image size, because RSA is applied only once (to encrypt the 32 byte AES key), and AES is well optimized comparing to RSA.

Decryption is always slower than encryption because d is much larger than $e = 65537$; our CRT implementation provides 4× speedup over naive $c^d \bmod n$. Task 2 is 2× slower than Task 1 because OAEP reduces the usable payload from $k-1$ to $k-66$ bytes per block, roughly doubling the block count.

The B&W vs. colored comparison confirms linear scaling: the ratio of timings closely matches the ratio of data sizes ($\frac{1555200}{193305} \approx 8$), as expected from the $O(N)$ block count dependency.

4. Security-Strength Evaluation

All metrics below are computed on BMP pixel data to ensure analysis operates on actual pixel values, since operating on the PNG file will study the compression DEFLATE stream rather then pixel values.

4.1. Visual & Histogram Analysis

Figure 2 shows that all three tasks produce visually random noise from the original butterfly image. However, the histograms reveal a crucial difference.

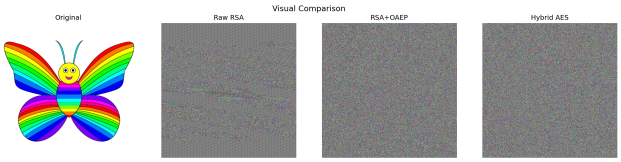

Figure 2: Original and encrypted images. All tasks produce visually random output.

Figure 3 shows the original image's RGB histograms: the distribution is highly non uniform, with strong peaks corresponding to the dominant colors in the butterfly. A good encryption algorithm is expected to flatten this distribution to approximate uniform randomness.

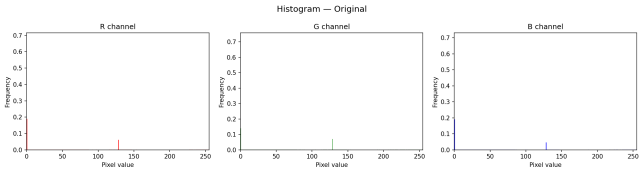

Figure 3: Original image: peaked, non-uniform pixel distribution across all channels.

The following histograms show how each task transforms the pixel value distribution.

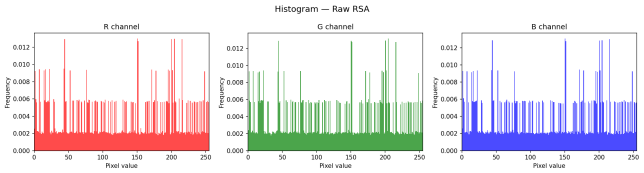

Figure 4: Task 1 (Raw RSA): visible spikes remain in the distribution. Deterministic RSA maps identical plaintext blocks to identical ciphertext, preserving statistical patterns from the original image.

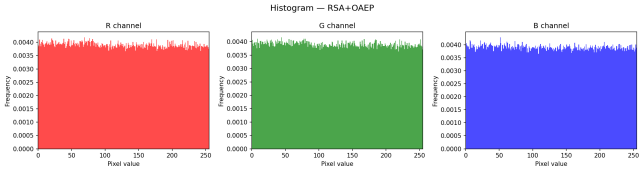

Figure 5: Task 2 (RSA+OAEP): near-uniform distribution. OAEP injects a random seed before each encryption, so identical blocks produce different ciphertexts, eliminating the pattern leakage seen in Task 1.

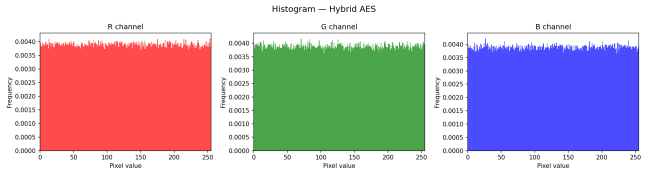

Figure 6: Task 3 (Hybrid AES): equally flat distribution as Task 2. AES-CBC's chaining mode propagates changes across blocks, achieving strong diffusion with near-ideal uniformity.

4.2. Entropy

Shannon entropy measures randomness per pixel. The theoretical maximum for 8-bit data is $H = 8.0$ bits.

Image	R	G	B	Avg
Colored butterfly				
Original	1.861	1.787	1.798	1.815
Task 1	7.733	7.734	7.734	7.734
Task 2	7.9995	7.9995	7.9996	7.9995
Task 3	**7.9997**	**7.9996**	**7.9996**	**7.9996**
B&W butterfly				
Original	5.404	5.434	5.403	5.414
Task 1	7.981	7.978	7.977	7.979
Task 2	7.997	7.997	7.997	7.997
Task 3	**7.997**	**7.997**	**7.997**	**7.997**

Table 3: Shannon entropy per channel. Tasks 2–3 approach ideal (8.0) on both images.

Task 1 achieves only 7.73 on the colored image because raw RSA is deterministic. The B&W image starts with higher base entropy 5.41 vs 1.82 due to its grayscale gradient, and Task 1 reaches 7.98 closer to ideal but still below Tasks 2–3.

4.3. Correlation Tests

Adjacent pixel correlation measures residual spatial structure. Ideal: ≈ 0.

Image	Horiz	Vert	Diag	Avg |r|
Colored butterfly (R channel)				
Original	0.975	0.942	0.928	0.948
Task 1	0.016	0.041	−0.050	0.036
Task 2	0.022	0.050	−0.021	0.031
Task 3	**−0.002**	**0.024**	**−0.009**	**0.012**
B&W butterfly (R channel)				
Original	0.935	0.936	0.889	0.920
Task 1	0.033	−0.022	−0.042	0.032
Task 2	−0.025	0.011	0.010	0.015
Task 3	**0.013**	**−0.008**	**0.021**	**0.014**

Table 4: Correlation coefficients. All tasks destroy spatial correlation; Task 3 is lowest.

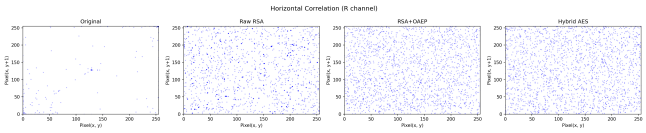

Figure 7: Correlation scatter (R, horizontal): original shows strong linear relationship; encrypted images show uniform scatter.

4.4. Chi-Squared & NPCR/UACI

The χ^2 test measures deviation from uniform distribution (ideal ≈ 255 the mean of the sample). NPCR and UACI measure sensitivity to plaintext changes (ideal NPCR $> 99.6\%$).

Image	χ^2 R	χ^2 G	χ^2 B
Original	66.4M	72.0M	68.9M
Task 1	223K	221K	221K
Task 2	348	326	321
Task 3	**214**	**287**	**253**

Task	NPCR %	UACI %
1	99.78	47.55
2	99.61	47.79
3	99.61	47.67

Table 5: χ^2 test (left) and NPCR/UACI (right) for colored image. Tasks 2–3 approach ideal $\chi^2 \approx 255$.

Task 1's $\chi^2 \approx 222$ K reflects the non-uniform histogram caused by deterministic RSA. Tasks 2–3 achieve $\chi^2 < 350$, very close to the expected value of 255 for a truly uniform distribution. All tasks exceed the 99.6% NPCR threshold.

4.5. Security Summary

Criterion	Ideal	Original	Task 1	Task 2	Task 3
Entropy	8.0	1.82	7.73	8.00	**8.00**
Corr |r|	0.00	0.95	0.04	0.03	**0.01**
χ^2	255	69M	222K	331	**251**
Semantic	✓	—	✗	✓	✓
Speed	Fast	—	Slow	Slower	**Fast**

Table 6: Overall comparison. Task 3 achieves the best result on every criterion.

5. Conclusion

Task 3 (Hybrid RSA+OAEP + AES-256-CBC) dominates across all dimensions: near ideal entropy (7.9996), lowest correlation (0.012), smallest χ^2 deviation (251), semantic security via OAEP, and a 20 000× speed advantage over pure RSA it didnt take an entire day to run like previous tasks. The performance confirms that Tasks 1–2 are impractical for real images: the colored BMP took over 60 minutes to decrypt with RSA alone. Task 3 completes the same work in 148 ms. This validates the standard of using asymmetric cryptography exclusively for key exchange while leaving the bulk encryption to symmetric ciphers like AES.